

Title: Effects of Data Augmentation in Computer Vision Algorithms

Research Question: How do the combinations of affine transformations, noise injection, and intensity transformation impact the accuracy and computational cost of a Convolutional Neural Network for object detection?

Subject: Computer Science

Session: May 2024

Word Count: 3882

Table of Contents

1. Introduction:	1
2. Background Information	3
2.1 Neural Networks	3
2.2 Convolutional Neural Networks	4
2.3 Data Augmentation Methods	5
2.4 Affine Transformations	5
2.4.1 Rotation	5
2.4.2 Translation	6
2.4.3 Scaling	7
2.4.4 Flip	8
2.5 Noise Injection	9
2.5.1 Gaussian Noise	9
2.6 Intensity Transformation	10
2.6.1 Brightness	10
2.6.2 Contrast	10
3. Methodology	11
3.1 Dataset	11
3.2 Classifier (Model) Architecture	12
3.3 Software	13
3.4 Variables	14
3.4.1 Independent Variables	14
3.4.2 Dependent Variables	14
3.4.3 Controlled variables and model specifications	15
3.5 Experimental Setup	16
3.6 Hypothesis	18
4. Results	18
4.1 Results Table for Primary Data	18
4.2 Graphical Results for Primary Dataset	19
4.3 Analyzing Primary Dataset Results	21
4.4 Results table for Secondary Dataset	22
4.5 Graphical Results for Secondary Dataset	22
4.6 Analyzing Secondary Data Results	24
4.7 Discussion of Primary and Secondary Data Results	24
5. Conclusion	26
6. Further research	27
7. Process Flow Diagram	27
8. Works Cited	29
9. Appendices	30

1. Introduction:

Machine Learning stands as a foundation of innovation in the rapidly evolving field of computer science. Machine Learning (ML) gives computers the power to learn patterns and make predictions from data without explicitly programming them. The shift from traditional programming methods to data-driven learning and targeted actions through data analysis has taken various industries by storm, including computer vision, where algorithms understand and interpret visual data.

One of the most important subsets of Computer Vision is object recognition algorithms that have transformed various industries such as cybersecurity, user authentication, and personalized experiences. The application of Object recognition ranges from unlocking smartphones to building rescue robots making it indispensable in our digital age. Object Recognition could be further improved by using Machine Learning models like Convolutional Neural Networks¹. CNNs are Neural Networks that are specialized for efficient image-processing tasks.

In spite of CNNs potential, they often face challenges when tackling the scarcity of diverse and large datasets. This scarcity is a potential cause of overfitting, where models become overly tailored to limited data hindering their ability to perform reliably across different scenarios that they weren't trained upon particularly. Hence, to unlock the full potential of CNNs in our increasingly data-driven and interconnected society addressing data limitations and mitigating overfitting are extremely important steps.

This is where Data augmentation steps in gaining prominence for their ability to improve model robustness and generalization capacity.

¹ IBM. "What Are Convolutional Neural Networks? | IBM." [Www.ibm.com](https://www.ibm.com/topics/convolutional-neural-networks), 2023, www.ibm.com/topics/convolutional-neural-networks.

Data augmentation methods add diversity to the dataset by manipulating the characteristics of the object in the image by flipping it or adding noise, amongst many other ways. This mitigates overfitting issues that can hinder the performance of object recognition algorithms. The techniques used for data augmentation could be neatly classified into three categories: Affine Transformations, Noise Injection, and Intensity transformations. A more comprehensive understanding of data augmentations' effect on an image could be obtained through the the picture below.

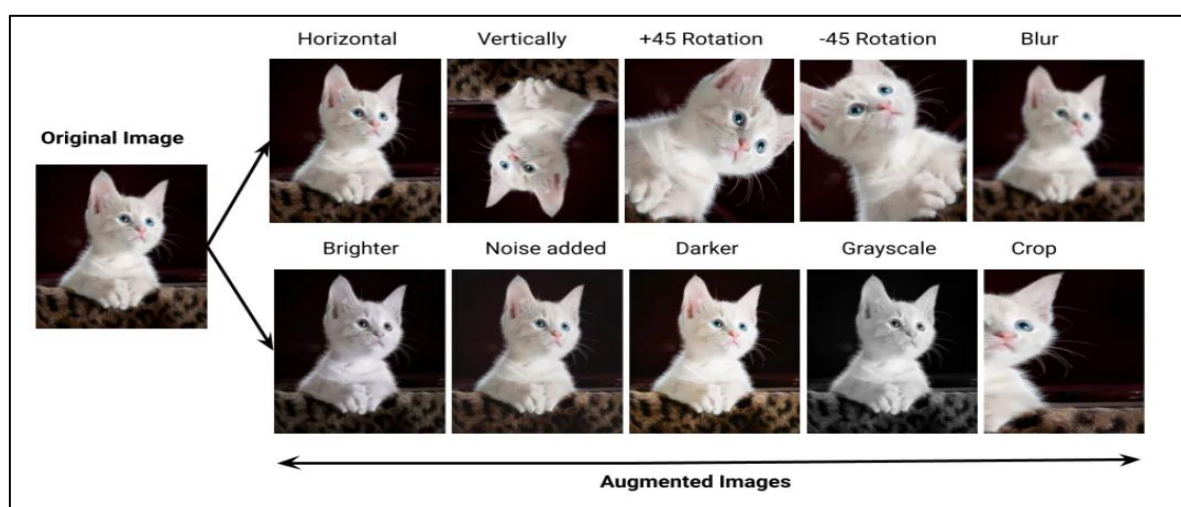


Figure 1: Representation of different data augmentation methods

Source: (<https://pranjal-ostwal.medium.com/data-augmentation-for-computer-vision-b88b818b6010>)

The advantages attached to the use of these methods make it imperative to explore what combinations of these methods could be used in unison (together) to get the best accuracy with the least computation cost for a CNN model. Hence, we reach the research question: **“How do the combinations of affine transformations, noise injection, and intensity transformation, impact the accuracy and computational cost of a Convolutional Neural Network for object detection?”**. This paper aims to uncover the best possible combinations eventually adding conclusive evidence for other researchers working on computer vision.

2. Background Information

To understand the effects of data augmentation methods in CNNs we first need to comprehend what are Neural Networks.

2.1 Neural Networks

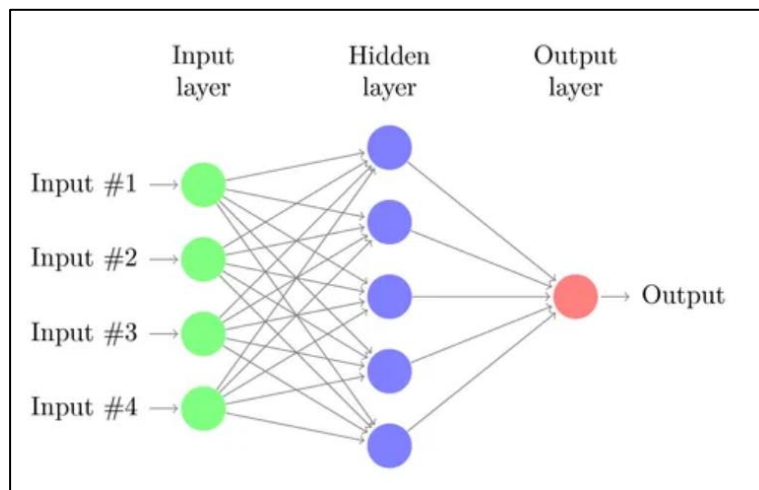


Fig 2: A Neural Network

Source: (<https://purnasaigudikandula.medium.com/a-beginner-intro-to-neural-networks543267bda3c8>)

The design of a Neural Network (NN) is inspired by the human brain. The circular dots in this representation are neurons that when grouped form a layer and when layers are grouped, they form the network. The input layer is where the data is inputted in the network, The hidden and the Output layer is where it is processed and the Output in our case will be the probability of the object being present in the image. Each Neuron is a specific set of algorithms that lets the network learn through the training data and create a function to predict results for new data. Each Neuron in the Neural Network uses a Gradient Descent² algorithm to optimize this function.

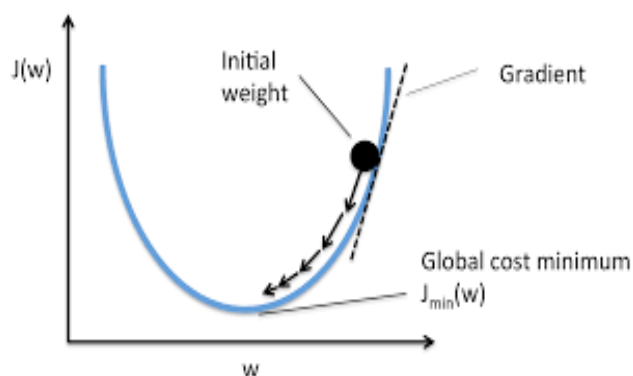


Fig 3: Gradient Descent

Source: (<https://sebastianraschka.com/faq/docs/gradient-optimization.html>)

² ("Gradient Descent - an Overview | ScienceDirect Topics")

2.2 Convolutional Neural Networks

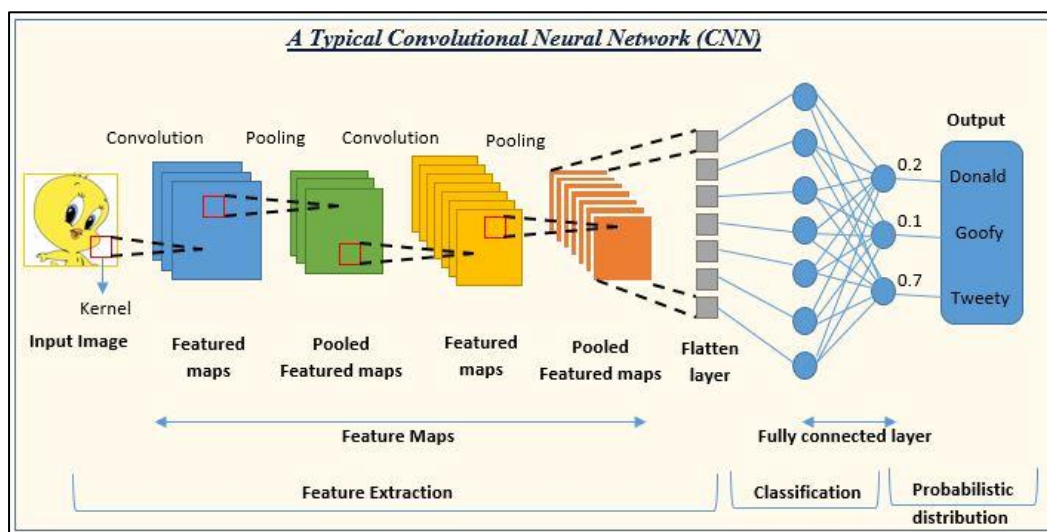


Figure 4: A typical Convolutional Neural Network

Source: (<https://www.analyticsvidhya.com/blog/2022/01/convolutional-neural-network-an-overview/>)

CNNs could be distinguished from Normal NNs due to their unique architecture. They have 3 main types of layers: Convolutional layer, Pooling layer, Fully-connected (FC) layer. Let's go through them one by one.

i) Convolutional layer: This is the core building block of CNNs. A Kernel also known as a filter or a feature detector is applied to a small area of the inputted data (image). The Kernel is a 2-D array which represents a feature of the image. The kernel moves across the entire image identifying that particular feature in the image and then creating a feature map. This could be done with multiple features using multiple feature detectors.

ii) Pooling layer: In this layer a pooling operation takes place which uses an average-taking algorithm to reduce the dimensionality (size) of the data to reduce computational cost. Another kernel slides across the feature maps to take the average and generate a smaller version of the map. There could be multiple Pooling and Convolutional layers in whatever combination is required breaking the image down to basic features before it is put into the Fully-connected layer.

iii) Fully-Connected layer: This layer in essence is a normal neural network that receives the broken-down data and generates the probability of the object in the image.

2.3 Data Augmentation Methods

As described earlier, Data augmentation is a pivotal technique in machine learning and computer vision that plays a crucial role in combating overfitting and enhancing model performance. For experimentation purposes, the methods were grouped into 3 different categories based on their characteristics, ending up in three main augmentation methods this makes them easier to evaluate without collisions between similar methods making the combinations clearer with fewer grey areas.

- Affine transformation (Rotation, Translation, Scaling & Flip)
- Noise removal/injection (Gaussian noise)
- Intensity transformation (Contrast & Brightness).

2.4 Affine Transformations

Affine Transformations³ include geometric manipulation of the object in the image. The 4 selected methods in this category are chosen as they involve straightforward operations that can help the model generalize better by exposing it to a broader range of object and image orientations during training.

2.4.1 Rotation

Rotation involves rotating an image from 0 to 360 degrees in either the left or right direction. A rotation matrix is multiplied by each coordinate to rotate an image sample counterclockwise.

³ "Affine Transformation." [www.mathworks.com, www.mathworks.com/discovery/affine-transformation.html#:~:text=Affine%20transformation%20is%20a%20linear](http://www.mathworks.com/discovery/affine-transformation.html#:~:text=Affine%20transformation%20is%20a%20linear).

$$(x_j, y_j, 1) = (x_i, y_i, 1) * \begin{pmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (1)$$

The positions of the particular coordinate before and after rotation are represented in the aforementioned equation⁴ by $(x_i, y_i, 1)$ and $(x_j, y_j, 1)$, respectively. If the angle is positive equation rotates in a counterclockwise direction to the stated degree and in a clockwise direction otherwise.



Figure 5: Original Image and rotated image

(The image is clicked by the author and transformed using OpenCV library's Rotate function)

2.4.2 Translation

With the use of a certain number of pixels, a picture can be translated along either the horizontal or vertical axis. Each coordinate is multiplied by a translation matrix⁵, much like with the rotation equation 1 (t_x and t_y control the translation in vertical(y) and horizontal direction(x) respectively)

⁴ Liu, Felix. "Understanding Transformations in Computer Vision." Medium, 4 Aug. 2021, towardsdatascience.com/understanding-transformations-in-computer-vision-b001f49a9e61.

⁵ "6.5 - Transformation Matrices — LearnWebGL." Learnwebgl.brown37.net, learnwebgl.brown37.net/transformations2/transformations_matrices.html. Accessed 26 Sept. 2023.

$$(x_j, y_j, 1) = (x_i, y_i, 1) * \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \quad (2)$$



Figure 6: Original Image and translated image

(The image is clicked by the author and transformed using OpenCV library's translate function)

2.4.3 Scaling

Scaling is an enhancement technique that either enlarges or reduces an image in size by a specified amount controlled through a scaling factor. Scaling is accomplished by using the following matrix equation:

$$(x_j, y_j, 1) = (x_i, y_i, 1) * \begin{pmatrix} k_x & 0 & 0 \\ 0 & k_y & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (3)$$

In the aforementioned equation, k_x represents the scaling factor in the x (horizontal) direction and k_y for the scaling factor in the y (vertical) direction. The matrix multiplication above is carried out for each coordinate, where $(x_i, y_i, 1)$ represents the coordinate's position before scaling and $(x_j, y_j, 1)$ represents the coordinate's position after scaling.



Figure 7: Original Image and scaled image

(The image is clicked by the author and transformed using OpenCV library's Scaling function)

2.4.4 Flip

Flipping, a data augmentation technique, operates by horizontally mirroring an image along its vertical axis. In this process, each pixel's position is inverted, essentially reflecting the image as if looking in a mirror. This transformation results in a new image that simulates a different viewpoint or orientation while maintaining the original content.

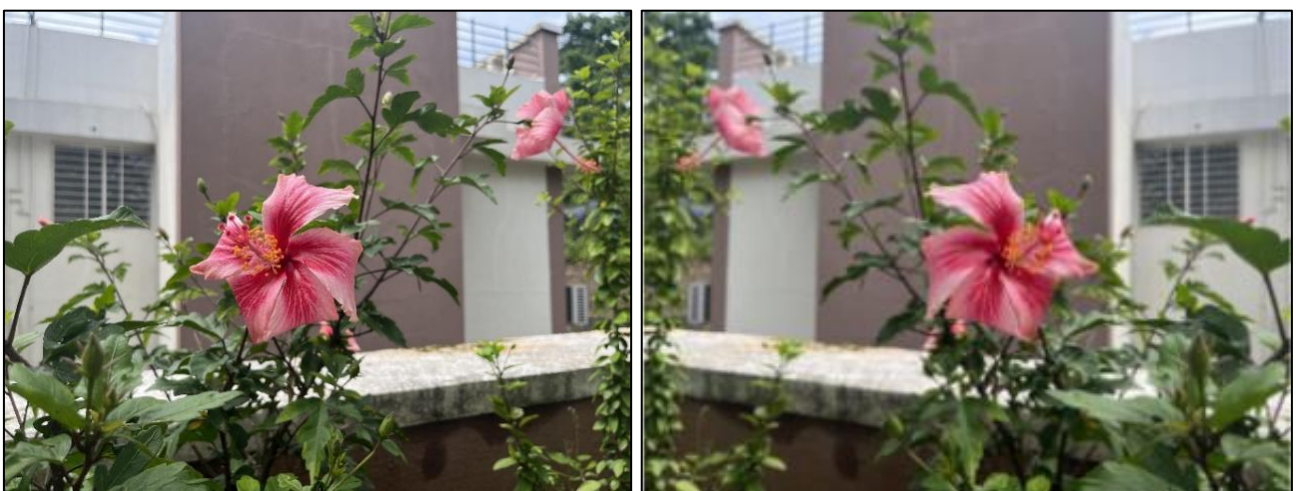


Figure 8: Original Image and flipped image

(The image is clicked by the author and transformed using OpenCV library's Flip function)

2.5 Noise Injection

Noise injection involves adding noise or blur to the existing image. This is advantageous in situations where the training images are sharp and clear but the test or realistic real-world data is more prone to errors by the collector making it lose its clarity. In such cases, Noise-injected images could bring the training data closer to the real-world data.

2.5.1 Gaussian Noise

The process of inputting a matrix of random values into image samples is called noise injection creating a blurred effect in the image. These numbers are taken from a Gaussian distribution, and the result is known as Gaussian Noise⁶. Its probability density function is:

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \times e^{\frac{-(x-\mu)^2}{2\sigma^2}} \quad (4)$$

where the distribution's mean μ , the standard deviation σ , and the random variable n (which has a normal distribution).



Figure 9: Original Image and Noise injected image

(The image is clicked by the author and transformed using OpenCV library's Gaussian Noise function)

⁶ "Gaussian_Noise." www.sfu.ca, www.sfu.ca/sonic-studio-webdav/handbook/Gaussian_Noise.html.

2.6 Intensity Transformation

Intensity transformation involves transforming the pixel intensity to change the brightness or the contrast of the image. This ability allows to increase the variations in terms of lighting conditions of the training dataset making the model more accurate when exposed to test data that has extreme lighting conditions.

2.6.1 Brightness

Brightness alteration is a data augmentation method that modifies the luminance or illumination of an image. When increasing brightness, each pixel's intensity values are incremented, causing the image to appear more illuminated. Conversely, reducing brightness involves decreasing the pixel values, resulting in a darker appearance.



Figure 10: Original Image and brightened image

(The image is clicked by the author and transformed using OpenCV library's Brightness function)

2.6.2 Contrast

The procedure can manipulate the contrast between an image's brightest and darkest areas. The pixel values are rescaled to emphasize the differences and boost contrast, enhancing the details of the image. The opposite of increasing contrast is diminishing it, resulting in a uniform appearance and a reduction in the visibility of smaller details.



Figure 11: Original Image and contrasted image

(The image is clicked by the author and transformed using OpenCV library's Contrast function)

3. Methodology

Several choices had to be made to determine what combinations of data augmentation approaches produced the maximum classification accuracy relative to their computational resource cost regarding the experiment's execution. Choosing a dataset is one of them. Approaches for augmentation, a classifier, and an experimental setup are provided in the listed sections below.

3.1 Dataset

In this study, the dataset used will be primary data collected by me. The dataset will then be stored in separate folders with the names of flowers present as labels on them. The primary dataset labels (Flower names) are Hibiscus, Madagascar Periwinkle, Morning Glory, Tecoma Capensis, and Thunbergia Erecta. Flowers were chosen as an object for this investigation due to fewer regulations on their data usage and the ready availability of varied data.

Moreover, for comparison, a large secondary dataset will also be used which is taken from the TensorFlow⁷ dataset of flowers coming from a collection of datasets that are ready to use, with

⁷ "TensorFlow Datasets." TensorFlow, www.tensorflow.org/datasets.

TensorFlow or other Python ML frameworks. The Secondary dataset labels are Daisy, Dandelion, Rose, Sunflower, and Tulip.

The Secondary dataset is made available as `tf.data.Datasets` (Appendix 9.2), enabling easy-to-use and high-performance input pipelines. This is primarily to see the effect of changing the size of the Dataset on the accuracy and computational cost of the algorithm. Below is the dataset directory structure.

```
Data
|
|
+---Primary Data
|   +---Hibiscus
|   +---Madagascar Periwinkle
|   +---Morning glory
|   +---Tecoma Capensis
|   \---Thunbergia Erecta
\---Secondary Data
    +---Daisy
    +---Dandelion
    +---Rose
    +---Sunflower
    \---Tulip
```

Fig 12: Data Directory Structure

The structure is created through the “Tree” Command in the windows PowerShell

3.2 Classifier (Model) Architecture

The classifier (model) used in the experiments was a simple CNN model to reduce computational complexity bringing uniformity to the analysis of results. The CNN has 3 Convolutional layers, 3 max pooling layers, a flattened layer, then the Fully connected layer with 128 neurons, and finally an output layer with neurons equal to the number of classes (5) present in the datasets. Also, dropout is utilized with a probability of 20% to decrease computational costs. Dropout was selected because it has been demonstrated to have an advantageous effect on the network's generalization capacity, which can enhance classification performance.

3 kernels or filters were selected for each max pooling and convolution layer to reduce the over-tailoring to training data since increasing their numbers could lead to overfitting as they hold the power to break data down to smaller bits making the model more adept at recognizing those small bits.

```
num_classes = 5

model = Sequential([
    layers.Conv2D(16, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dropout(0.2),
    layers.Dense(128, activation='relu'),
    layers.Dense(num_classes)
])
```

Figure 13: Classifier architecture

Source: (The classifier architecture is created by the author in Google Colab IDE)

3.3 Software

This experiment was conducted through Google Colab⁸, an online Python Integrated Development Environment (IDE) providing a sophisticated and powerful means to develop machine learning solutions using Jupyter notebooks. The model was created using an open-source machine learning library called Tensorflow which was also used for implementing the data augmentation methods. Open CV⁹ was another library used for preprocessing the data.

⁸ "Google Colaboratory." Colab.research.google.com, colab.research.google.com/?utm_source=scs-index.

⁹ OpenCV. "OpenCV Library." Opencv.org, 2019, opencv.org/.

3.4 Variables

3.4.1 Independent Variables

The independent variables used in this experiment are a combination of data augmentation methods belonging to the categories described above.

Number	Variable or combination
1.	Baseline (No Augmentation)
2.	Noise injection (N)
3.	Intensity transformations (I)
4.	Affine transformations (A)
5.	Affine transformations and Noise injection (A and N)
6.	Affine transformations and Intensity transformations (A and I)
7.	Noise injection and Intensity transformation (N and I)
8.	Upper bound (all augmentation)

Table 1: Independent variable or augmentation method combinations

3.4.2 Dependent Variables

The effect of changing the independent variables will be assessed using the following dependent variables.

Dependent Variables	Description
Time elapsed	The time it took for the CNN model to complete training using the Python time () function.
Accuracy	The CNN model's accuracy on the test set a using the evaluate() function contained in the TensorFlow library. This is calculated after the model is been trained on the training set

Table 2: Dependent Variables

Note on Accuracy Measurement: The accuracy is measured by using this formula

$$\frac{\text{No. of Correct predictions}}{\text{No. of All Prediction}}$$

Accuracy of 1 means that the model has 100% accuracy.

Source: <https://www.javatpoint.com/how-to-check-the-accuracy-of-your-machine-learning-model>

3.4.3 Controlled variables and model specifications

Variable	Value/How it's controlled	Function	Affects which Dependent Variable
Optimizer	Adam	Controls the process of gradient descent	Affects resource management and performance
Loss Functions	Binary – Cross entropy	Value that is optimized throughout training	performance
Processor GPU and RAM	RAM: 12.7 GB GPU: Tesla T4	Determines the time taken to perform functions	performance
Python and keras version	Python 3.12 and keras 2.14	Different python and keras version trains with different performances and accuracies	Performance and Resource management.
IDE Used	Google Colab	Some IDE's are slow than others	Performance
Imports and Dependencies	Appendix 9.1 details all imports and their version	Different versions of libraries may have different dependencies	Performance and resource management
Final epochs	10	Determines how many weights and updates happen	Performance and resource management

Table 3: Controlled variables and model specifications

3.5 Experimental Setup

The primary dataset of 250 images was split into training and test datasets 70% training, 15% validation, and 15% test, and the secondary dataset of 3670 images was split in the same way too. The datasets are visualized in the tables below.

Split	Size	Samples per Class
Training	175	35
Validation	38	7
Test	37	8

Table 4: The split of train and test set for primary data

Split	Size	Samples per Class
Training	2569	514
Validation	550	110
Test	551	110

Table 5: The split of train and test set for secondary data

To make the data format constant the images were preprocessed and resized to 180*180 pixels to save computational cost. This was done using OpenCV¹⁰'s resize method. The RGB values were kept to increase the variation in the dataset. Moreover, each image was scaled by dividing every pixel value by 255 again to save computational cost. The dataset was dynamically augmented to improve the network's generalizability using the TensorFlow layers Module¹¹. (Refer to Appendix 9.3).

¹⁰ OpenCV. "OpenCV Library." Opencv.org, 2019, opencv.org/.

¹¹ "Module: Tf.keras.layers | TensorFlow Core R2.1." TensorFlow, 2020, www.tensorflow.org/api_docs/python/tf/keras/layers.

Utilizing this strategy also saves storage space compared to augmenting the data, storing it on the disk, and then putting it in the model. This feat was achieved by creating a pipeline of augmentation methods and creating different Sequential models out of these categories containing the combinations. An example is given in the figure below.

```
#Affine Transformations
affine_transformations = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.2),
    layers.RandomTranslation(0.2,0.2),
    layers.Rescaling(0.2)
])

#Noise_injection
noise_injection = keras.Sequential([
    layers.GaussianNoise(0.2)
])

#Affine and Noise
A_N = keras.Sequential([
    layers.GaussianNoise(0.1),
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
    layers.Rescaling(0.2)
])
```

Figure 14: Example of Implementation of Augmentation methods in code

Source: (The combinations is created by the author in Google Colab IDE)

The model was then trained on the Training datasets and tested through the test datasets. Next, the results for accuracy and computational cost were collected. The obtained results were then analyzed in detail through illustrative Bar Charts and Graphs created through the Matplotlib library¹².

¹² Matplotlib. "Matplotlib: Python Plotting — Matplotlib 3.1.1 Documentation." Matplotlib.org, 2012, matplotlib.org/.

3.6 Hypothesis

It can be Hypothesized that the upper bound which is all transformations applied at once will achieve the best results as it will create the greatest number of images and the most varied training dataset among all combinations judging by the fact that an increased number of training examples and variance in the data decreases overfitting. This result could be expected for both the primary dataset and the secondary dataset. In terms of computational cost, all augmentations will be the most expensive with Noise injection having the lowest computational cost as it has the lowest augmentation methods inside the category as seen in section 2.5.

4. Results

The effectiveness of each augmentation method combination is assessed in terms of its influence on classification accuracy and its computational resource cost (in this experiment, running time), as was previously indicated in section 3.4. Under section 4.1, the study's key findings for the primary dataset are provided with an illustrative table and bar charts. The results are further detailed in section 4.2 about the influence on classification accuracy, the cost of computational resources, and relative performance.

4.1 Results Table for Primary Data

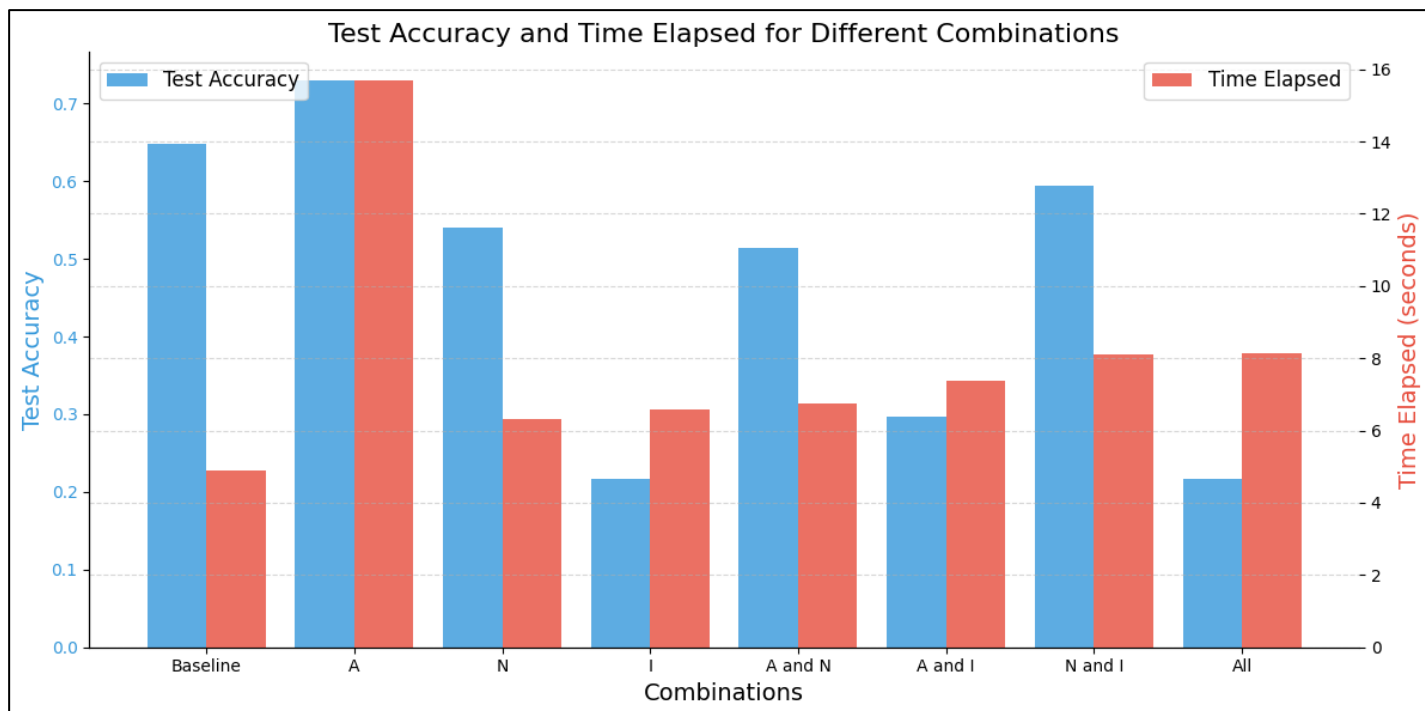
This section contains the Raw results gained from the experiment carried out. Validation Accuracy is not included in the table because the hyperparameters remained constant throughout the experiment thus their Validation accuracy and Validation loss are similar to test accuracy and test loss which made validation accuracy redundant. The display of Validation dataset loss alongside training dataset loss is a great measure for overfitting since if both constantly reduce we can conclude reduction in overfitting. Hence, it is displayed on the graphs.

4.1 Results Table Primary Dataset

Combination	Training Loss	Training Accuracy	Validation Loss	Test loss	Test Accuracy	Time Elapsed (seconds)
Baseline	0.1039	0.9767	0.6263	0.9708	0.6486	4.890
Affine (A)	0.893	0.7965	0.4360	0.6281	0.7297	15.69
Noise (N)	0.4308	0.8605	0.7982	1.0001	0.5405	6.305
Intensity(I)	1.435	0.2733	1.547	1.542	0.2162	6.569
Affine and Noise	0.5873	0.7616	1.012	1.576	0.5135	6.7409
Affine and Intensity	1.402	0.2791	1.363	1.395	0.2973	7.368
Noise and Intensity	0.334	0.8895	0.9525	1.701	0.5946	8.113
All augmentation	1.4027	0.2791	1.447	1.445	0.2162	8.128

Table 6: Experimental results for Primary Dataset

4.2 Graphical Results for Primary Dataset



Bar Graph 1: Test Accuracy and Time Elapsed Against combination for Primary dataset



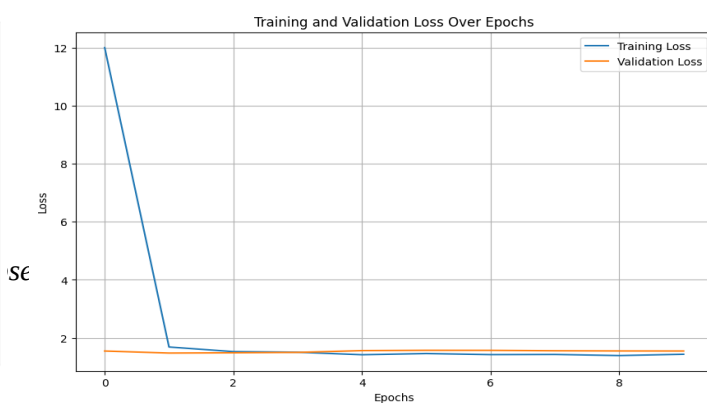
Graph 1: No transformations



Graph 2: Affine transformations



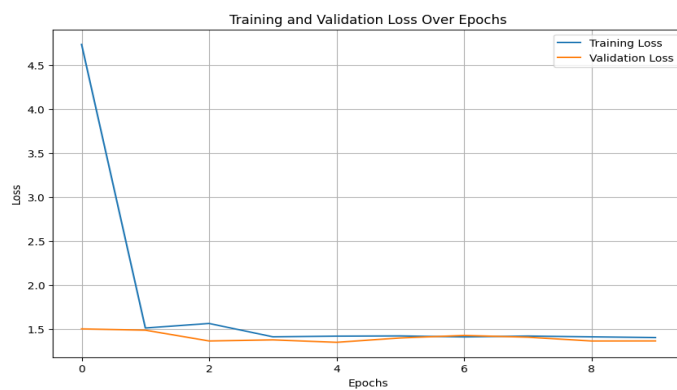
Graph 3: Noise Injection



Graph 4: Intensity Transformation



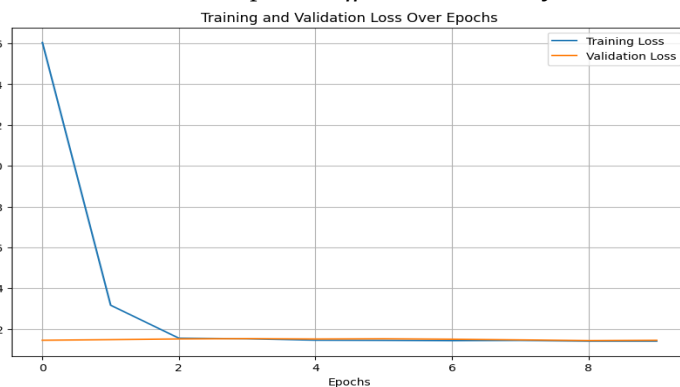
Graph 5: Affine and Noise



Graph 6: Affine and Intensity



Graph 7: Noise and Intensity



Graph 8: All Transformations

4.3 Analyzing Primary Dataset Results

The obtained data indicates that applying different data augmentation methods does have a great influence on the computational cost and accuracy of the model.

Accuracy

The combination of data augmentation methods which produced the highest accuracy of about 73% which is 8% more than the baseline was affine transformations as indicated in the Bar chart. From looking at Graph 2 which shows Loss vs Epochs for affine transformations we can see that the data is not overfitting as both the validation and training data are decreasing continuously after every epoch. Thus, applying affine transformations does solve the overfitting problem. The combination of Affine and Intensity transformation along with all augmentations resulted in worse classification accuracy than the baseline of 22% and 29% accuracy respectively which are the worst accuracy received by any combination of methods. On close inspection of Table 6, we can see a positive correlation between affine transformations and classification accuracy as whenever it is included even with a bad-performing augmentation method still it increases the accuracy as seen in the cases of intensity coupled with affine and noise coupled with affine.

Computation or Run-time cost

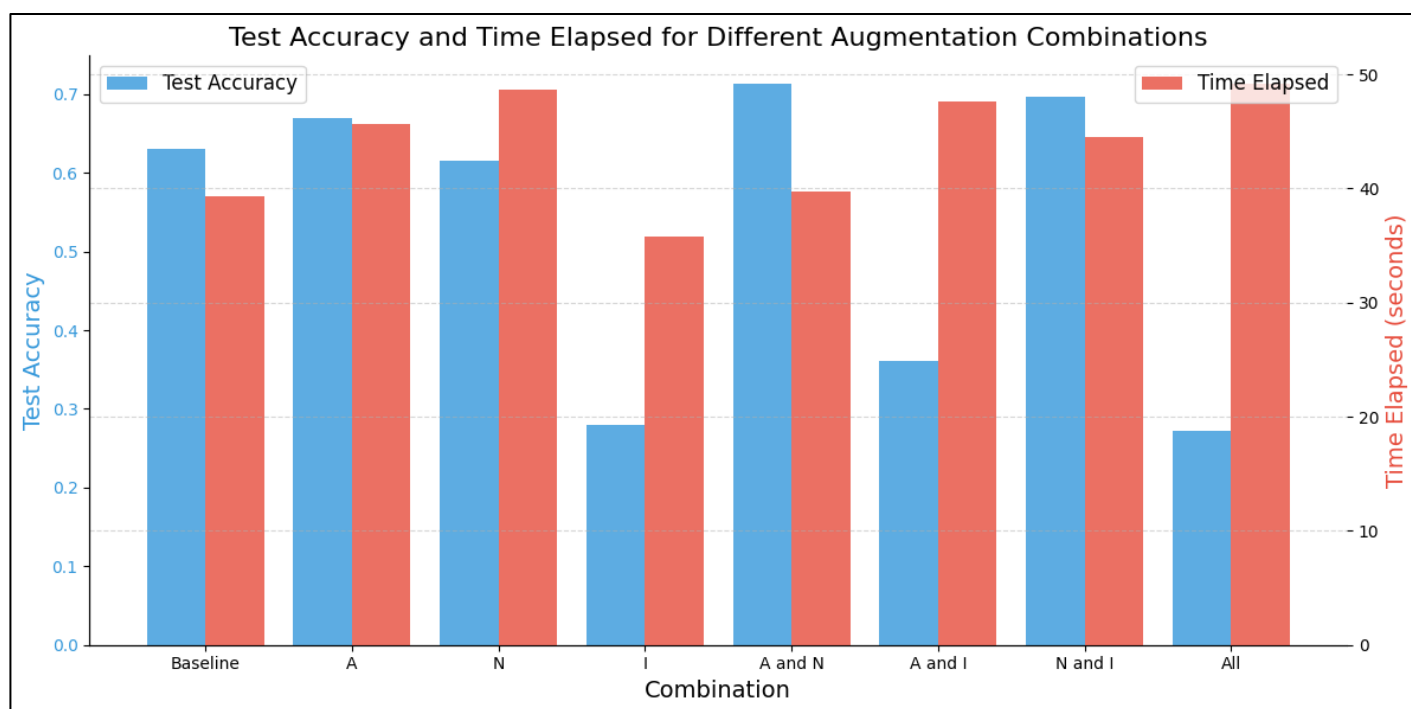
Looking at the run-time cost or the computational cost in table 6, the natural answer comes to light the baseline or the regular dataset with no augmentation methods applied performs the best with the time elapsed in training the model of 4.9 seconds. The best performer in classification accuracy affine transformations resulted in the most computational cost with a time elapsed of 15.7 seconds. However, the other combination fits perfectly in the series where the natural answer for the highest computation cost is all augmentation methods combined or the upper bound. Affine transformation being the most computation cost inefficient is surprising as all augmentation methods combined are the most intuitive answer.

4.4 Results table for Secondary Dataset

Combination	Training Loss	Training Accuracy	Validation Loss	Test loss	Test Accuracy	Time Elapsed (seconds)
Baseline	0.0227	0.9969	2.213	2.225	0.6297	39.27
Affine (A)	0.8499	0.6784	0.9318	0.8861	0.6697	45.68
Noise (N)	0.1221	0.9615	1.678	1.817	0.615	48.66
Intensity (I)	1.602	0.2452	1.602	1.591	0.2794	35.76
Affine and Noise	0.7385	0.7084	0.8364	0.7801	0.7132	39.71
Affine and Intensity	1.609	0.2476	1.5316	1.153	0.3611	47.57
Noise and Intensity	0.7291	0.7193	0.8198	0.8013	0.6969	44.53
All augmentation	1.602	0.2409	1.603	1.595	0.2722	49.17

Table 7: Experimental results for Secondary Data

4.5 Graphical Results for Secondary Dataset



Bar Graph 2: Test Accuracy and Time Elapsed Against combination for Secondary dataset



Graph 9: No transformation



Graph 10: Affine Transformation



Graph 11: Noise injection



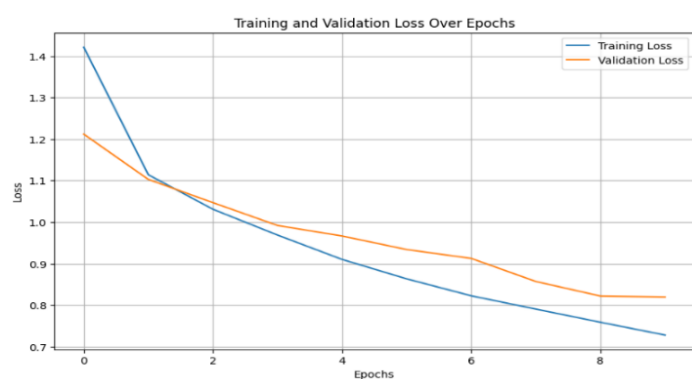
Graph 12: Intensity Transformation



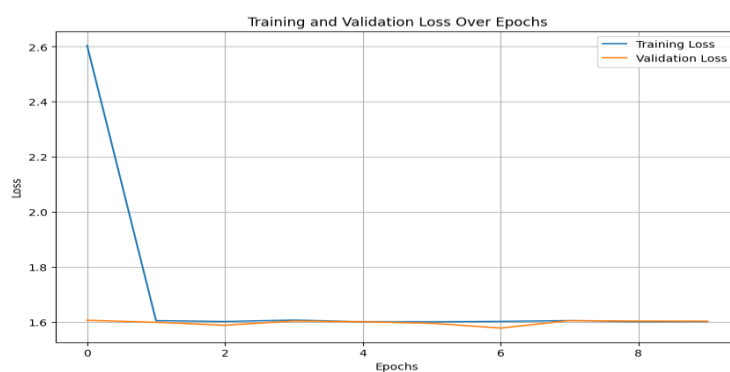
Graph 13: Affine and Noise



Graph 14: Affine and Intensity



Graph 15: Noise and Intensity



Graph 16: All Transformation

4.6 Analyzing Secondary Data Results

Accuracy

The obtained data from experimentation on the secondary dataset reflects that Affine transformation and Noise injection had the best classification accuracy of 71% as seen in Table 7 and Bar Chart 2. Looking at Graph 13, we can see that the augmentation method addresses overfitting as the validation and training loss both decrease continuously with epochs. As seen with Primary data results we see a positive correlation between affine transformation and classification accuracy. However, combinations containing Intensity (Intensity, Affine, Intensity, and All augmentations) except Noise and Intensity transformations performed the worst scoring 28%, 36%, and 28% on Test Accuracy respectively. This shows a negative correlation between Intensity transformation and Test Accuracy.

Computation or Run-time cost

As discussed earlier while analyzing the primary data experimentation results, the obvious answer to the most computational cost efficient will be the baseline with no augmentation methods applied achieving a time elapsed while training of the model of about 39.3 seconds. In comparison to the Primary data experimentation, the data obtained shows a clear relation between the number of data augmentation techniques applied and the computational cost as the most expensive data augmentation method was all augmentation with 49.7 seconds as seen in Bar Chart 2. It is reasonable that training the model with the greatest number of augmentation methods takes the most time due to many operations taking place.

4.7 Discussion of Primary and Secondary Data Results

The discussion of primary and secondary data experimentation together is imperative because both datasets were of different sizes. The secondary dataset which is 15 times larger than the primary dataset shows the differences when the augmentation methods are applied to datasets of different sizes.

If we take the difference in results between the two, we can see that the best performing in terms of accuracy for the primary dataset was just affine transformation whereas for the secondary it was affine transformation and noise injection. This could potentially stem from the fact that the primary data might not have been as diverse and varied as it was collected by me while the secondary dataset was collected by experts. Most likely this occurred due to the challenges I faced while capturing pictures inside the vicinity of my school where there isn't much diversity between flower appearances.

If we look at the similarity between the result there is one significant pattern observable is that affine transformation has a positive impact on the classification accuracy. This is also in accord with the findings of Le Cun et.al¹³ and Ellias Bonnici et.al.¹⁴ which show that implementing affine transformation methods on computer vision models increases classification accuracy.

On close inspection, we can see that Intensity transformation augmentation methods hurt the classification accuracy as it is the worst performing method for both primary and secondary data together. Furthermore, if intensity transformation is introduced with any other augmentation method it reduces the overall classification accuracy as seen in the primary data when only affine transformation has 73% accuracy but when coupled with intensity transformation it goes down to 29.3% accuracy on the test set. If we look at the other dependent variable in our research computational or run time cost, we see a trend in secondary data with increasing augmentation methods applied the computational cost increases. However, in the primary data results, affine transformation has the highest computational cost which is the only data point out of trend and thus should be treated as an anomaly.

¹³ LeCun, Yann, Kavukcuoglu, Koray, and Farabet, Clement. "Convolutional networks and applications in vision". In: Proceedings of 2010 IEEE International Symposium on Circuits and Systems. 2010, pp. 253–256.

¹⁴ Bonnici, Elias, and Per Arn. "The Impact of Data Augmentation on Classification Accuracy and Training Time in Handwritten Character Recognition."

5. Conclusion

In this research, the effects of applying different augmentation method combinations on a CNN model accuracy and model training time were studied graphically and numerically. The research conducted was successful in identifying that the application of data augmentation has a positive impact on classification accuracy. However, the usage of the data augmentation method increases computation time. Therefore, it is of great significance to select augmentation methods that increase classification accuracy while maintaining a feasible computation cost. Hence, the relative computational cost of applying data augmentation was also analyzed.

The study has identified that applying affine transformation has a positive impact on the classification accuracy of object detection models. This is also supported by studies conducted by Le Cun et.al and Ellias Bonnici et.al. However, the initial hypothesis proved to be wrong in the case where applying all augmentation methods did not perform the best.

On the other hand, the initial hypothesis was proved to be correct in the case of run-time or computational cost as the most time-consuming augmentation method was all augmentation for secondary data as well as primary data experimentation results except for one anomaly in the primary results of affine transformations. This anomaly might have occurred due to the limitation in collecting the primary dataset.

Looking at the results we can also conclude that the data augmentation method applied should be selected after a good understanding of the dataset has been made to understand whether or not noise or intensity transformations should be introduced while affine transformations should be used as a must-have for computer vision application for enhanced classification accuracy.

Conceivably, with the results of this research, it could be proven that this research will prove to be vital for deep learning developers to make data augmentation choices for their object detection algorithms and contribute to the Artificial intelligence field at large.

6. Further research

It would be interesting to see the effects of these augmentation methods combinations on the other datasets such as a dataset for faces. Moreover, other types of neural network architectures such as a recurrent neural network or transfer learning methods such as VGG16 or VGG 19¹⁵ could also be used to see the impact of augmentation on the classification accuracy. Another good addition to the experimentation could be the inclusion of elastic distortion and morphological deformation methods such as dilation and erosion adding another level of depth to the research with an increased amount of combination to experiment with thus yielding better data to analyze.

7. Process Flow Diagram

A process flow diagram is given on the next page to illustrate the stages of work conducted during the experiment.

¹⁵ Team, Keras. "Keras Documentation: VGG16 and VGG19." Keras.io, keras.io/api/applications/vgg/.

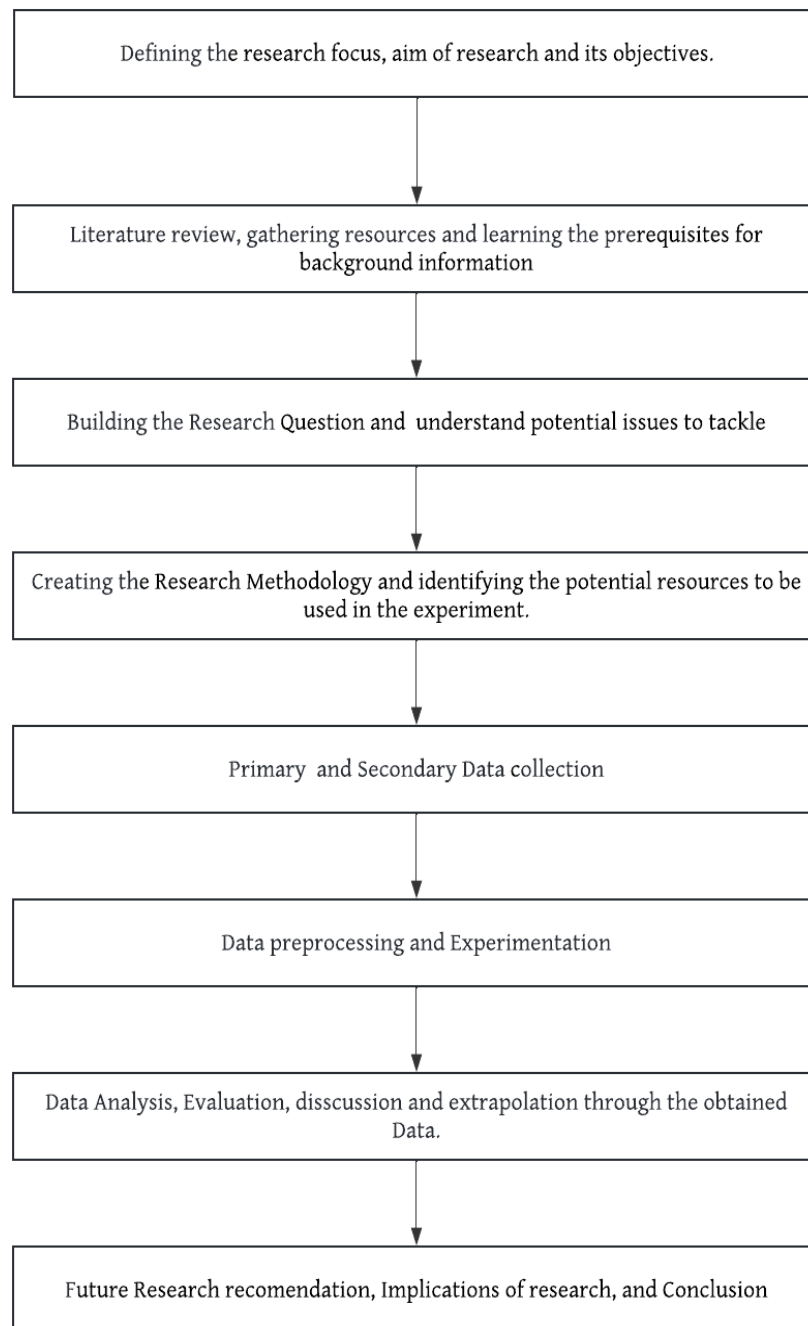


Fig 15: Process Flow Diagram

8. Works Cited

1. “6.5 - Transformation Matrices — LearnWebGL.” *Learnwebgl.brown37.net*, learnwebgl.brown37.net/transformations2/transformations_matrices.html. Accessed 26 Sept. 2023.
2. “Affine Transformation.” *Www.mathworks.com*, www.mathworks.com/discovery/affine-transformation.html#:~:text=Affine%20transformation%20is%20a%20linear.
3. Bonnici, Elias, and Per Arn. “The Impact of Data Augmentation on Classification Accuracy and Training Time in Handwritten Character Recognition.”
4. “Gaussian_Noise.” *Www.sfu.ca*, www.sfu.ca/sonic-studio-webdav/handbook/Gaussian_Noise.html.
5. “Google Colaboratory.” *Colab.research.google.com*, colab.research.google.com/?utm_source=scs-index.
6. “Gradient Descent - an Overview | ScienceDirect Topics.” *Www.sciencedirect.com*, www.sciencedirect.com/topics/computer-science/gradient-descent. Accessed 18 Jan. 2024.
7. IBM. “What Are Convolutional Neural Networks? | IBM.” *Www.ibm.com*, 2023, www.ibm.com/topics/convolutional-neural-networks.
8. Team, Keras. “Keras Documentation: VGG16 and VGG19.” *Keras.io*, keras.io/api/applications/vgg/.
9. Liu, Felix. “Understanding Transformations in Computer Vision.” *Medium*, 4 Aug. 2021, towardsdatascience.com/understanding-transformations-in-computer-vision-b001f49a9e61.

10. LeCun, Yann, Kavukcuoglu, Koray, and Farabet, Clement. “Convolutional networks and applications in vision”. In: Proceedings of 2010 IEEE International Symposium on Circuits and Systems. 2010, pp. 253–256.
11. Matplotlib. “Matplotlib: Python Plotting — Matplotlib 3.1.1 Documentation.” *Matplotlib.org*, 2012, matplotlib.org/.
12. “Module: Tf.keras.layers | TensorFlow Core R2.1.” *TensorFlow*, 2020, www.tensorflow.org/api_docs/python/tf/keras/layers.
13. OpenCV. “OpenCV Library.” *Opencv.org*, 2019, opencv.org/.
14. “OpenCV: Geometric Transformations of Images.” *Docs.opencv.org*, docs.opencv.org/3.4/da/d6e/tutorial_py_geometric_transformations.html.
15. “TensorFlow Datasets.” *TensorFlow*, www.tensorflow.org/datasets.

9. Appendices

9.1 Imports and Dependencies

```
import matplotlib.pyplot as plt
import numpy as np
import cv2
import os
import PIL
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential
from google.colab.patches import cv2_imshow
import time
```

Dependency	Version
Matplotlib	3.8.2
Numpy	1.23.5
OpenCV	4.8.0
Pillow	9.4.0
Tensorflow	2.15.0

Table 8: Dependencies and Versions

9.2 Importing and Preprocessing Secondary Data

```
dataset_url = "https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz"
data_dir = X_train_scaled = X_train / 255 if dataset_url, cache_dir='/content/drive/MyDrive/CS EE /Secondary Data', untar=True)
# calculate X_test_scaled = X_test / 255 .ch means current directory
# untar X_val_scaled = X_val / 255
```

```
from sklearn.model_selection import train_test_split

# First, split the data into a training set (70%) and a temporary set (30%)
X_train, X_temp, y_train, y_temp = train_test_split(X, y, random_state=0, train_size=0.70, test_size=0.30)

# Then, split the temporary set into a validation set (20%) and a test set (20%)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, random_state=0, test_size=0.5)
```

```
flowers = ["roses", "daisy", "dandelion", "sunflowers", "tulips"]
flowers_images_dict = {}
for flower in flowers:
    flowers_images_dict[flower] = list(data_dir.glob(flower+'/*'))
```

```
flowers_labels_dict = {
    'roses':0,
    'daisy':1,
    'dandelion':2,
    'sunflowers':3,
    'tulips':4
}
```

```
X, y = [], []
for flower_name, images in flowers_images_dict.items():
    for image in images:
        img = cv2.imread(str(image))
        resized_img = cv2.resize(img, (180,180))
        X.append(resized_img)
        y.append(flowers_labels_dict[flower_name])
```

9.3 Defining the Augmentation Methods

```
#Affine Transformations
affine_transformations = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.2),
    layers.RandomTranslation(0.2,0.2),
    tf.keras.layers.Rescaling(0.2)
])

#Noise_injection
noise_injection = keras.Sequential([
    layers.GaussianNoise(0.1)
])

#Intensity Transformation
intensity_transformations = keras.Sequential([
    layers.RandomBrightness(0.1),
    layers.RandomContrast(0.1)
])
```

9.4 Importing and preprocessing Primary Data

```
data_dir = "/content/drive/MyDrive/CS EE /Primary Data"
```

```
import pathlib
data_dir = pathlib.Path(data_dir)
data_dir
```

```
PosixPath('/content/drive/MyDrive/CS EE /Primary Data')
```

```
image_count = len(list(data_dir.glob('*/*.jpeg')))
image_count
```

```
295
```

```
Hibiscus = list(data_dir.glob('Hibiscus/*'))
len(Hibiscus)
PIL.Image.open(str(Hibiscus[2]))
```

```
names = ["Hibiscus", "Madagascar Perwinkle", "Morning glory", "Tecoma capensis", "Thunbergia erecta"]
images_dict = {}
for name in names:
    images_dict[name] = list(data_dir.glob(name+'/*'))
```

```
flower_labels_dict = {
    'Hibiscus':0,
    'Madagascar Perwinkle':1,
    'Morning glory':2,
    'Tecoma capensis':3,
    'Thunbergia erecta':4,
}
```

```
X, y = [], []
for name, images in images_dict.items():
    for image in images:
        img = cv2.imread(str(image))
        resized_img = cv2.resize(img, (180, 180))
        X.append(resized_img)
        y.append(flower_labels_dict[name])
```

```
from sklearn.model_selection import train_test_split

# First, split the data into a training set (70%) and a temporary set (30%)
X_train, X_temp, y_train, y_temp = train_test_split(X, y, random_state=0, train_size=0.70, test_size=0.30)

# Then, split the temporary set into a validation set (15%) and a test set (15%)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, random_state=0, test_size=0.5)

X_train_scaled = X_train / 255
X_test_scaled = X_test / 255
X_val_scaled = X_val / 255
```

9.5 Classifier Architecture

```
num_classes = 5

model = Sequential([
    layers.Conv2D(16, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(32, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, padding='same', activation='relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(num_classes)
])

model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

start_time = time.time()
model_history = model.fit(X_train_scaled, y_train, epochs=10, validation_data=(X_val_scaled, y_val))
end_time = time.time()
training_time = end_time - start_time
print(f"Training time: {training_time}")
```

9.6 Graph Plotting Function

```
import matplotlib.pyplot as plt

def plot_training_loss(history):
    """
    Plot the training and validation loss over epochs.

    Args:
        history: Model training history containing training and validation loss information.
    """
    plt.figure(figsize=(10, 6))
    plt.plot(history.history['loss'], label='Training Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.title('Training and Validation Loss Over Epochs')
    plt.legend()
    plt.grid(True)
    plt.show()
```